

TECHNICAL EXECUTION ROADMAP

Constraint-Aware Multi-Day Itinerary Planner

with Time Windows

1-Year Academic Roadmap

Table of Contents

Table of Contents.....	2
Executive Summary	5
Core Principles of This Roadmap.....	5
Technology Stack Overview.....	5
Phase Summary	5
1. Goals.....	6
2. Core Concepts to Learn.....	6
3. Implementation Tasks	6
3.1 OSM Data Ingestion Pipeline	6
3.2 PostGIS Spatial Query Layer	6
3.3 FastAPI Backend Skeleton.....	7
3.4 React + Leaflet Frontend Prototype.....	7
4. Recommended Architecture	7
5. Risks and Pitfalls	7
Common Mistakes	8
6. Validation Strategy	8
7. Expected Demo by End of Phase 1	8
8. Resume / Interview Value Added.....	8
9. Estimated Difficulty	9
10. What Would Make This Phase Elite.....	9
Weekly Breakdown — Phase 1	9
1. Goals.....	11
2. Core Concepts to Learn.....	11
3. Implementation Tasks	11
3.1 OSRM Integration and Travel-Time Matrix	11
3.2 Core Data Structures	11
3.3 Feasibility Checker.....	12
3.4 Insertion Heuristic	12
4. Edge Cases and Failure Scenarios	13
5. Testing Strategy	13
5.1 Unit Tests.....	13
5.2 Integration Tests	14
6. Validation Strategy	14
7. Expected Demo by End of Phase 2	14
8. Resume / Interview Value Added.....	14

9. Estimated Difficulty	14
10. What Would Make This Phase Elite	15
Weekly Breakdown — Phase 2	15
1. Goals	17
2. Core Concepts to Learn	17
3. Implementation Tasks	17
3.1 Score Function Design	17
3.2 2-opt Local Search	18
3.3 Or-opt Local Search	18
3.4 OR-Tools Exact Solver for Benchmarking	18
4. Evaluation Framework	18
4.1 Test Instance Generator	19
4.2 Evaluation Metrics	19
4.3 Complexity Analysis	19
5. Risks and Pitfalls	19
6. Visualization Goals	20
7. Expected Demo by End of Phase 3	20
8. Resume / Interview Value Added	20
9. Estimated Difficulty	20
10. What Would Make This Phase Elite	21
Weekly Breakdown — Phase 3	21
1. Goals	23
2. Core Concepts to Learn	23
3. Implementation Tasks	23
3.1 Frontend Polish	23
3.2 Itinerary Explanation Engine	23
3.3 Final Architecture Cleanup	24
4. Documentation Goals	24
4.1 README.md (Portfolio Centrepiece)	24
4.2 Final Report Structure	24
5. What Interviewers and Professors Will Ask	25
6. How to Present Tradeoffs Honestly	25
7. Demo Scenarios	25
Scenario A: Happy Path	25
Scenario B: Constrained Edge Case	26
Scenario C: Evaluation Demo	26

8. Portfolio-Quality GitHub Repository	26
9. Resume / Interview Value Added.....	26
10. What Would Make This Phase Elite	26
Weekly Breakdown — Phase 4	27
Cross-Phase Summary & Dependencies	28
Phase Dependencies	28
Minimum Viable vs. Stretch Goals	28
Final Correctness Checklist	28
A Final Word on Engineering Philosophy.....	29

Executive Summary

This document presents a rigorous, 1-year technical execution roadmap for building a Constraint-Aware Multi-Day Itinerary Planner. The system generates optimized multi-day travel itineraries subject to daily time budgets, POI opening hours, travel-time constraints, and user preference scores.

The roadmap is structured into four progressive phases spanning an academic year. Each phase is designed to be implementable by a student team while producing portfolio-quality work that demonstrates real algorithmic depth and engineering discipline.

Core Principles of This Roadmap

Engineering Philosophy — Non-Negotiable
Correctness over feature count: A working feasibility checker beats a broken ML recommender every time.
Evaluation over buzzwords: Measure optimality gaps, not marketing claims.
Engineering depth over fake complexity: Clean algorithms > microservices theater.
Feasibility over hype: Do not claim globally optimal routing — no one does at this scale.

Technology Stack Overview

Component	Technology
Spatial Data	OpenStreetMap + PostGIS (PostgreSQL)
Routing Engine	OSRM (Open Source Routing Machine)
Backend API	FastAPI (Python)
Frontend	React + Leaflet.js
Exact Solver (Benchmarking)	Google OR-Tools
Core Optimization	Insertion Heuristic + 2-opt / Or-opt Local Search

Phase Summary

Phase	Focus Area & Duration
Phase 1	Foundations & Spatial Infrastructure — Weeks 1–10
Phase 2	Core Routing Engine — Weeks 11–21
Phase 3	Optimization & Evaluation — Weeks 22–32
Phase 4	Productization & Final Presentation — Weeks 33–42

PHASE 1

Foundations & Spatial Infrastructure

Duration: Weeks 1–10 (approx. 10 weeks)

1. Goals

Phase 1 establishes the entire technical foundation. Every subsequent phase depends on the correctness of what is built here. The goal is a spatial database loaded with real POI data, a working candidate extraction pipeline, a skeleton API, and a frontend capable of map-based region selection. No optimization logic is built yet.

2. Core Concepts to Learn

- PostGIS spatial extensions: geometry types, ST_Within, ST_Distance, ST_DWithin, spatial indexes
- OpenStreetMap data model: nodes, ways, relations, tags (amenity=museum, tourism=attraction, etc.)
- Overpass API and osmium-tool for OSM data extraction and filtering
- EPSG coordinate systems: WGS84 (EPSG:4326) vs projected coordinate systems for distance accuracy
- FastAPI async endpoints, Pydantic models, dependency injection
- React component lifecycle, state management, Leaflet integration
- Docker Compose for local service orchestration (Postgres, OSRM, API)

3. Implementation Tasks

3.1 OSM Data Ingestion Pipeline

- Download regional PBF extract from Geofabrik (e.g., city or country level)
- Use osmium-tool to filter to amenity/tourism/leisure nodes only
- Write a Python ingestion script using psycopg2 to insert POIs into PostGIS
- Schema: poi_id, name, category, lat, lon, geometry (POINT), opening_hours_raw, tags (JSONB)
 - Parse OSH (opening_hours) strings using the opening_hours.js library or a Python equivalent
 - Store parsed hours as structured time windows: day_of_week, open_time, close_time
- Create spatial index: CREATE INDEX poi_geom_idx ON pois USING GIST(geometry)

3.2 PostGIS Spatial Query Layer

- Implement bounding-box query: extract all POIs within a user-drawn polygon

- Implement category filter: allow filtering by POI type (museum, restaurant, park, etc.)
- Implement radius query: find POIs within N km of a point using ST_DWithin
- Write integration tests that validate row counts and geometry correctness

3.3 FastAPI Backend Skeleton

- GET /pois?bbox=... — returns POIs within bounding box with category filters
- GET /pois/{id} — returns full POI detail including parsed opening hours
- POST /itinerary — placeholder endpoint, returns mock data for now
- Implement proper Pydantic response models with type annotations throughout
- Add OpenAPI docs at /docs — this is a deliverable in itself

3.4 React + Leaflet Frontend Prototype

- Render OSM tile layer (OpenStreetMap or MapTiler free tier)
- Implement Leaflet.draw plugin for polygon/rectangle region selection
- On selection: call /pois API, render POI markers with popup cards
- Implement category filter sidebar with checkboxes
- Allow user to add POIs to a 'trip list' (local state only at this stage)
- Display POI name, category, opening hours in marker popup

4. Recommended Architecture

Layer	Detail
Database	PostgreSQL 15 + PostGIS 3.x in Docker container
Data Loader	Python CLI script: ingest_osm.py — run once, idempotent
API	FastAPI with SQLAlchemy Core (not ORM — direct SQL for spatial queries)
Frontend	React 18, Vite bundler, Leaflet 1.9, Axios for API calls
Dev Environment	Docker Compose: services for db, api, and frontend dev server

5. Risks and Pitfalls

⚠ CRITICAL: What NOT to Build in Phase 1
• Do NOT implement any optimization logic — that is Phase 2 and 3.
• Do NOT build a user authentication system — it wastes weeks and adds zero algorithmic value.
• Do NOT use an ORM for spatial queries — raw SQL with PostGIS functions is clearer and faster.

- | |
|--|
| <ul style="list-style-type: none"> Do NOT try to parse all opening_hours formats perfectly — handle 80% of cases, log failures. |
| <ul style="list-style-type: none"> Do NOT deploy to a cloud service yet — local Docker is sufficient and keeps iteration fast. |

Common Mistakes

- Using EPSG:4326 (degrees) for ST_Distance and getting distances in degrees, not meters. Fix: use ST_DistanceSphere or cast to a metric projection.
- Forgetting to create a GIST spatial index, causing full-table scans on every polygon query.
- Loading the entire country PBF when only a city is needed — this costs hours of load time.
- Not validating opening_hours parsing: malformed strings silently drop time-window data.
- Over-designing the React state management — plain useState is fine for Phase 1.

6. Validation Strategy

- Load 10,000+ POIs for a known city (e.g., London, Berlin, Paris). Verify count against known OSM data.
- Run bounding-box query for a known neighborhood. Manually verify a sample of 20 results on the OSM website.
- Verify spatial index is used: EXPLAIN ANALYZE your polygon query must show 'Index Scan' not 'Seq Scan'.
- Verify opening hours are parsed correctly for 10 manually inspected POIs by cross-checking against Google Maps.
- API response time for a city-level polygon query must be under 500ms.
- Frontend must render 200+ POI markers without freezing the browser (use Leaflet cluster plugin if needed).

7. Expected Demo by End of Phase 1

A user can open the web app, draw a rectangle over a city district, select categories (museums, restaurants, parks), and see POI markers appear on the map with correct name, category, and opening hours. The POI data is live from the PostGIS database.

8. Resume / Interview Value Added

Portfolio Signal — Phase 1
PostGIS spatial queries: rare and impressive on a student resume.
OSM data pipeline: shows you work with real, messy, production-scale geographic data.
Docker Compose environment: demonstrates DevOps maturity without overengineering.

- Clean API design with OpenAPI docs: professional engineering habit.

9. Estimated Difficulty

Task	Difficulty
PostGIS setup and spatial indexing	Medium — documentation is good, but spatial SQL is new to most students
OSM ingestion pipeline	Medium-High — data quality issues require robust error handling
Opening hours parsing	High — the opening_hours spec is complex; time-box this to 1 week
FastAPI skeleton	Low — well-documented, fast to build
React + Leaflet frontend	Medium — Leaflet.draw configuration is fiddly

10. What Would Make This Phase Elite

- Write a correctness test that runs ST_Contains on every returned POI to verify it actually lies within the drawn polygon — catches edge cases in geometry handling.
- Build a data quality dashboard: show what percentage of ingested POIs have valid opening hours, names, coordinates — demonstrates engineering maturity.
- Implement a tile-based POI clustering query that scales gracefully to 50,000+ POIs.
- Write a one-command setup script (make setup) that spins up the entire environment from scratch.

Weekly Breakdown — Phase 1

Week	Focus
Week 1	Environment setup: Docker Compose, PostgreSQL + PostGIS, verify spatial functions work
Week 2	OSM data download, osmium filtering, Python ingestion script v1
Week 3	Schema design, spatial indexing, opening_hours parsing (basic cases)
Week 4	Spatial query functions: bbox, radius, polygon — with integration tests
Week 5	FastAPI skeleton: /pois, /pois/{id} endpoints, Pydantic models, OpenAPI docs
Week 6	React project setup, Leaflet map render, tile layer, basic marker display
Week 7	Leaflet.draw integration, polygon selection triggers API call, markers populate
Week 8	Category filter sidebar, POI popup cards with opening hours display
Week 9	Trip list state (add/remove POIs), data quality validation, index verification

Week	Focus
Week 10	Buffer week: fix technical debt, write tests, prepare Phase 1 demo

PHASE 2**Core Routing Engine***Duration: Weeks 11–21 (approx. 11 weeks)***1. Goals**

Phase 2 is the algorithmic heart of the project. The goal is to take a list of user-selected POIs and produce a feasible multi-day itinerary that respects opening hours, daily time budgets, and travel time constraints. Correctness is the only metric that matters in this phase. An itinerary that violates a single time window constraint is a failure.

2. Core Concepts to Learn

- OSRM HTTP API: /table endpoint for travel-time matrix, /route for individual routes
- Time-window constraint scheduling: earliest arrival, latest start, propagation logic
- Insertion heuristics: cheapest insertion, greedy insertion, regret-based insertion
- Feasibility checking: forward time propagation through a route sequence
- Data structure design for itinerary representation
- Unit and property-based testing for algorithmic code

3. Implementation Tasks**3.1 OSRM Integration and Travel-Time Matrix**

- Deploy OSRM in Docker using the same regional OSM extract from Phase 1
- Implement a Python client for the OSRM /table endpoint
- Input: list of (lat, lon) coordinates; Output: N×N matrix of travel times in seconds
- Cache matrix computations to avoid redundant OSRM calls
- Validate against known routes: OSRM London–Paris should match realistic drive time

3.2 Core Data Structures

These structures must be designed before writing any algorithm. Get them right — changing them midway is expensive.

Structure	Description
POI	id, name, lat, lon, category, time_windows: List[TimeWindow], visit_duration_minutes
TimeWindow	day_of_week, open_time (datetime.time), close_time (datetime.time)
ItinerarySlot	poi, day_index, arrival_time, departure_time, travel_time_from_prev

Structure	Description
DayPlan	day_index, slots: List[ItinerarySlot], total_time, feasibility_status
Itinerary	days: List[DayPlan], total_score, total_travel_time, feasible: bool
TravelMatrix	poi_ids: List[str], times: np.ndarray (N×N seconds)

3.3 Feasibility Checker

The feasibility checker is the most critical component. Every other algorithm depends on it being correct.

Pseudocode: Forward Time Propagation
- function check_feasibility(day_plan, travel_matrix, day_start_time, daily_budget): - current_time = day_start_time - for i, slot in enumerate(day_plan.slots): - if i > 0: - travel = travel_matrix[prev_poi_id][slot.poi.id] - current_time += travel - window = get_window_for_day(slot.poi, day_plan.day_index) - if current_time > window.close_time - visit_duration: return INFEASIBLE - if current_time < window.open_time: current_time = window.open_time # wait - slot.arrival_time = current_time - current_time += slot.poi.visit_duration - slot.departure_time = current_time - if current_time - day_start_time > daily_budget: return INFEASIBLE - return FEASIBLE

3.4 Insertion Heuristic

The insertion heuristic builds an itinerary greedily by inserting POIs one at a time at the position that causes the least increase in total travel time while maintaining feasibility.

Pseudocode: Cheapest Feasible Insertion
- function build_itinerary(pois, travel_matrix, config): - itinerary = empty_itinerary(config.num_days) - unassigned = sort_pois_by_preference_score(pois, descending=True)

• for poi in unassigned:
• best_position = None; best_cost = infinity
• for day in itinerary.days:
• for position in possible_insertion_positions(day):
• candidate = insert_at(day, poi, position)
• if check_feasibility(candidate, travel_matrix, config):
• cost = insertion_cost(day, poi, position, travel_matrix)
• if cost < best_cost:
• best_cost = cost; best_position = (day, position)
• if best_position is not None:
• apply_insertion(itinerary, poi, best_position)
• else:
• mark_unschedulable(poi)
• return itinerary

4. Edge Cases and Failure Scenarios

 Edge Cases That Must Be Handled Explicitly
• POI closed on the trip day: must skip or schedule on a different day — never silently produce infeasible plan.
• Travel time exceeds daily budget alone: POI cannot fit regardless of position — mark as unschedulable.
• Waiting time causes time budget violation: arriving early at a POI may push the day over budget.
• Empty travel matrix row: OSRM can return null for unreachable nodes — handle gracefully.
• Zero visit_duration: must default to a minimum (e.g., 30 minutes) to avoid degenerate schedules.
• All POIs on same day violates budget: system must distribute across multiple days automatically.

5. Testing Strategy

5.1 Unit Tests

7. Feasibility checker: construct a route that violates opening hours — assert INFEASIBLE.

8. Feasibility checker: construct a route with exactly one minute to spare — assert FEASIBLE.
9. Insertion cost function: verify triangle inequality holds for test travel matrices.
10. Travel matrix builder: verify symmetry ($A \rightarrow B$ time $\approx B \rightarrow A$ time for road networks).

5.2 Integration Tests

11. Load 5 real POIs from the Phase 1 database, generate travel matrix from OSRM, run insertion heuristic, verify output is a valid Itinerary object with no constraint violations.
12. Test with deliberately impossible input (6 POIs all closing at 17:00, daily budget = 2 hours) — system must return a partial but feasible itinerary with unscheduled POI list.

6. Validation Strategy

13. For every generated itinerary: run the feasibility checker as a post-generation validator. It must never return FEASIBLE for a plan that violates constraints.
14. Manual spot-check: for 3 generated itineraries, calculate arrival times by hand and verify they match the system output.
15. Stress test: generate itineraries for 50 random POI sets. Zero tolerance for constraint violations.
16. Performance: generating a 10-POI, 3-day itinerary must complete in under 2 seconds.

7. Expected Demo by End of Phase 2

A user selects 10–15 POIs on the map, specifies 3 days and 8 hours per day. The system returns a feasible multi-day itinerary displayed as a sequenced list per day with arrival times, visit durations, and travel times between stops. Every constraint is respected.

8. Resume / Interview Value Added

Portfolio Signal — Phase 2
Time-window constraint propagation: directly relevant to interview problems on scheduling (Google, Meta, Amazon all ask these).
Insertion heuristic for routing: a textbook algorithm you can explain precisely, with complexity analysis.
OSRM integration: shows experience with real-world routing infrastructure.
Property-based testing of algorithmic correctness: distinguishes strong engineers.

9. Estimated Difficulty

Task	Difficulty
OSRM Docker deployment	Low-Medium — well-documented but large data files
Data structure design	Medium — getting it right before coding saves significant rework
Feasibility checker	High — subtle off-by-one errors in time propagation are common
Insertion heuristic	High — correctness requires careful handling of all edge cases
Test coverage	Medium — writing good adversarial test cases requires domain thinking

10. What Would Make This Phase Elite

- Implement a formal constraint violation report: for every infeasible insertion attempt, log which constraint was violated (time window, travel time, budget) — enables debugging and generates evaluation data.
- Profile the insertion heuristic and identify the bottleneck (likely the $O(n^2)$ feasibility check per insertion). Document it — this shows performance awareness.
- Add a regret-based insertion variant: instead of always inserting the highest-preference POI, insert the POI with the highest regret (largest gap between best and second-best insertion positions). This is a real improvement and easy to explain in interviews.

Weekly Breakdown — Phase 2

Week	Focus
Week 11	OSRM Docker setup, test /table endpoint manually with Postman, validate travel times
Week 12	Design and implement all core data structures; code review with team before proceeding
Week 13	Implement travel-time matrix builder with caching; unit tests
Week 14	Implement TimeWindow parsing from PostGIS data; validate against 20 real POIs
Week 15	Implement feasibility checker; unit tests including all edge cases
Week 16	Implement insertion heuristic v1 (no local search yet); integration test
Week 17	Connect insertion heuristic to POST /itinerary endpoint; return structured JSON
Week 18	Frontend: display itinerary as day-by-day list; show arrival/departure times
Week 19	Adversarial testing: impossible inputs, edge time windows, zero-budget scenarios
Week 20	Performance profiling; fix any $O(n^3)$ or worse bottlenecks in heuristic
Week 21	Buffer: fix bugs, complete test coverage, prepare Phase 2 demo

PHASE 3

Optimization & Evaluation

Duration: Weeks 22–32 (approx. 11 weeks)

1. Goals

Phase 3 transforms the feasible-but-unoptimized heuristic into a demonstrably strong solution. The dual goals are: (1) improve solution quality through local search, and (2) rigorously evaluate how good the solutions are through benchmarking against OR-Tools exact solver. This phase produces publishable-quality evaluation results.

2. Core Concepts to Learn

- 2-opt local search: swap two edges in a route to reduce total travel time
- Or-opt local search: move one or three consecutive POIs to a different position
- Score functions: combining travel time, preference score, and waiting time penalties
- OR-Tools CP-SAT solver: constraint programming for small exact instances
- Optimality gap: $(\text{heuristic_cost} - \text{optimal_cost}) / \text{optimal_cost} \times 100\%$
- Experimental design: controlling variables, replicability, statistical significance
- Complexity analysis: time and space complexity of each algorithm

3. Implementation Tasks

3.1 Score Function Design

Before implementing optimization, define what you are optimizing. The score function must be explicit and documented.

Score Function Components
• <code>preference_score</code>: sum of user-assigned scores for all scheduled POIs (maximize)
• <code>travel_time_penalty</code>: total travel time in minutes (minimize)
• <code>waiting_time_penalty</code>: total waiting at closed POIs in minutes $\times$ <code>penalty_weight</code> (minimize)
• <code>coverage_bonus</code>: number of unique categories covered (maximize)
• Final score = <code>preference_score</code> - <code>alpha</code> $\times$ <code>travel_time_penalty</code> - <code>beta</code> $\times$ <code>waiting_time_penalty</code> + <code>gamma</code> $\times$ <code>coverage_bonus</code>
• Recommended starting weights: <code>alpha</code>=0.3, <code>beta</code>=0.5, <code>gamma</code>=10. Tune empirically.

3.2 2-opt Local Search

Pseudocode: 2-opt for Single Day	
•	function two_opt(day_plan, travel_matrix, config):
•	improved = True
•	while improved:
•	improved = False
•	for i in range(1, len(day_plan.slots) - 1):
•	for j in range(i+1, len(day_plan.slots)):
•	new_plan = reverse_segment(day_plan, i, j)
•	if check_feasibility(new_plan, travel_matrix, config):
•	if score(new_plan) > score(day_plan):
•	day_plan = new_plan; improved = True; break
•	if improved: break
•	return day_plan

3.3 Or-opt Local Search

- Or-opt(1): move a single POI from its current position to every other position in the same day or a different day
- Or-opt(3): move a chain of 3 consecutive POIs together
- Apply Or-opt after 2-opt for further improvement — the two moves complement each other
- Implement a move acceptance criterion: only accept moves that improve score AND maintain feasibility

3.4 OR-Tools Exact Solver for Benchmarking

The OR-Tools integration exists solely for benchmarking. It is NOT part of the production system. Limit to instances with $n \leq 10$ POIs per day — beyond this, exact solving is intractable.

- Model as a CP-SAT problem: binary variables for assignment, integer variables for arrival times
- Constraints: time windows, travel times, daily budget
- Objective: maximize preference score minus travel time penalty
- Set a solve time limit of 60 seconds per instance
- Log: optimal_score, solve_time, whether proven optimal or timed out

4. Evaluation Framework

4.1 Test Instance Generator

Generate controlled test instances to measure algorithm performance systematically.

- Small (n=5): 5 POIs, 1 day — OR-Tools can find provably optimal solution
- Medium (n=10): 10 POIs, 2 days — OR-Tools at the boundary of tractability
- Large (n=20): 20 POIs, 3 days — heuristic only, use lower bound as reference
- Generate 50 random instances per size. Fix random seed for reproducibility.

4.2 Evaluation Metrics

Metric	Definition and Purpose
Optimality Gap (%)	$(\text{heuristic_cost} - \text{optimal_cost}) / \text{optimal_cost} \times 100$. Core quality metric.
Schedule Improvement (%)	$(\text{score_after_local_search} - \text{score_before}) / \text{score_before} \times 100$
Feasibility Rate (%)	% of generated itineraries that pass the feasibility checker
Runtime (ms)	Wall-clock time for heuristic and local search at each n
POI Coverage (%)	% of input POIs successfully scheduled across all days
Waiting Time (min)	Total time spent waiting for POIs to open — lower is better

4.3 Complexity Analysis

Algorithm	Time Complexity
Travel matrix construction	$O(n^2)$ — n OSRM API calls, each $O(1)$
Insertion heuristic	$O(n^2 \times k)$ where k = feasibility check cost ($O(n)$)
2-opt per iteration	$O(n^2)$ — all pairs, $O(n)$ feasibility check each
2-opt until convergence	$O(n^3)$ worst case — $O(n^2)$ per improvement $\times O(n)$ improvements
Or-opt(1) per iteration	$O(n^2 \times d)$ where d = number of days
OR-Tools (exact)	Exponential in worst case — tractable only for $n \leq 10$

5. Risks and Pitfalls

 Evaluation Pitfalls — These Will Fail Your Viva
• Comparing heuristic against OR-Tools on instances where OR-Tools did NOT prove optimality — invalid gap calculation. • Using the same instances for tuning score weights AND for reporting results — data contamination.

Claiming your heuristic 'approaches optimal' without specifying the instance size — always report n.
Not reporting runtime — a solution that takes 10 minutes is not useful regardless of quality.
Local search that sometimes produces worse solutions than the insertion heuristic — indicates a scoring bug.

6. Visualization Goals

- Optimality gap bar chart: x-axis = instance size, y-axis = gap (%), with error bars for standard deviation across 50 instances
- Runtime scaling plot: log-log plot of n vs. runtime for heuristic and OR-Tools
- Score improvement chart: before/after 2-opt and Or-opt for each instance size
- Map overlay: show itinerary route as a Leaflet polyline with day-color coding
- Schedule timeline: horizontal Gantt-style chart showing POI visits and travel times per day

7. Expected Demo by End of Phase 3

A comprehensive evaluation report is produced showing: (1) the heuristic generates feasible solutions 100% of the time; (2) for small instances, the optimality gap is measured and reported; (3) local search improves the insertion heuristic solution by $X\%$ on average; (4) runtime scales predictably with instance size. All results are reproducible from a single script.

8. Resume / Interview Value Added

Portfolio Signal — Phase 3
Optimality gap measurement: shows statistical rigor that impresses hiring panels.
2-opt / Or-opt local search implementation: classic combinatorial optimization, asked in FAANG interviews.
OR-Tools integration: demonstrates experience with professional constraint programming tools.
Experimental design: designing a fair, reproducible benchmark shows research-level thinking.
Complexity analysis: being able to discuss $O(n^3)$ worst-case vs. observed behavior is interview gold.

9. Estimated Difficulty

Task	Difficulty
Score function design and tuning	Medium — requires empirical experimentation
2-opt with feasibility checking	Medium-High — common bugs in segment reversal logic
Or-opt moves	High — multi-day Or-opt requires careful feasibility re-propagation
OR-Tools CP-SAT model	High — constraint modeling requires careful variable design
Evaluation framework and statistics	Medium — straightforward once test generator is built

10. What Would Make This Phase Elite

- Implement a lower bound estimator for large instances where OR-Tools cannot prove optimality. Even a loose lower bound (e.g., LP relaxation or assignment relaxation) makes the gap analysis more credible.
- Run a sensitivity analysis on score function weights (alpha, beta, gamma) and show how solution quality varies — demonstrates understanding of the multi-objective nature of the problem.
- Implement a restart strategy for local search: run from 5 different starting insertion orders and take the best result. This is a simple improvement that has real impact and is easy to explain.

Weekly Breakdown — Phase 3

Week	Focus
Week 22	Design and implement score function; unit tests verifying all components
Week 23	Implement 2-opt for single-day plans; validate on toy instances by hand
Week 24	Extend 2-opt to multi-day (intra-day only); integration tests
Week 25	Implement Or-opt(1); test against 2-opt on 20 benchmark instances
Week 26	OR-Tools CP-SAT model for small instances ($n \leq 10$); verify correctness on 5 known cases
Week 27	Build test instance generator; generate 50×3 instance sets (small/medium/large)
Week 28	Run full evaluation: heuristic + local search on all instances; log all metrics
Week 29	Run OR-Tools on small/medium instances; compute optimality gaps
Week 30	Produce evaluation charts: gap bar chart, runtime plot, score improvement chart

Week	Focus
Week 31	Implement Leaflet route polyline; Gantt timeline visualization in frontend
Week 32	Buffer: refine results, fix anomalies, prepare Phase 3 demo

PHASE 4

Productization & Final Presentation

Duration: Weeks 33–42 (approx. 10 weeks)

1. Goals

Phase 4 transforms a working research prototype into a polished, presentable system. The goals are: professional frontend UX, comprehensive documentation, a compelling viva/demo presentation, and a portfolio-quality GitHub repository that communicates the project's technical depth to any reader in under 5 minutes.

2. Core Concepts to Learn

- Technical writing: how to communicate algorithms to a mixed audience
- System documentation: README quality standards, architecture diagrams
- Demo engineering: building demos that don't fail under pressure
- Tradeoff presentation: how to honestly discuss what your system cannot do
- Git repository hygiene: commit history, branch strategy, release tagging

3. Implementation Tasks

3.1 Frontend Polish

- Day cards: each day rendered as a card with a numbered POI list, arrival/departure times, and total travel time
- Color-coded route polylines: each day's route rendered in a distinct color on the map
- POI detail drawer: click any POI to see full details (opening hours, visit duration, preference score)
- Itinerary summary panel: total trip score, total travel time, total waiting time, POI coverage %
- Loading states and error messages: user-facing feedback for all API calls
- Responsive layout: must work on both desktop and tablet screen widths

3.2 Itinerary Explanation Engine

For each generated itinerary, produce a human-readable explanation. This is NOT an AI feature — it is simple template-based generation.

- 'Day 1 starts at British Museum (opens 10:00). 25-minute walk to National Gallery. Closes at 18:00 — you arrive at 14:30 with 90 minutes to spare.'
- Highlight any waiting times: 'Note: Tate Modern opens at 10:00. Arriving at 09:30 means 30 minutes of waiting.'

- Flag unscheduled POIs with reason: 'The Tower of London could not be included — adding it would exceed your Day 2 time budget by 45 minutes.'

3.3 Final Architecture Cleanup

- Audit all API endpoints: remove unused routes, standardize error response format
- Add request validation: reject malformed inputs with informative 400 responses
- Write a one-command Docker Compose deployment: docker compose up must give a running system
- Remove all TODO comments, dead code, and debug print statements
- Pin all dependency versions in requirements.txt and package.json

4. Documentation Goals

4.1 README.md (Portfolio Centrepiece)

The README is the first thing any recruiter or professor reads. It must communicate the project in 3 minutes.

17. Project title and one-paragraph summary: what it does, why it is hard, what your approach is.
18. Architecture diagram (draw.io or Excalidraw): show all system components and data flow.
19. Quick start: three commands to get the system running locally.
20. Algorithm section: explain insertion heuristic + local search with a concrete example.
21. Evaluation results: include the optimality gap chart and key numbers inline.
22. Limitations section: be honest about what the system does not handle.

4.2 Final Report Structure

Section	Content
Abstract	Problem, approach, key results in 250 words
1. Introduction	Motivation, problem statement, contributions
2. Related Work	VRPTW literature, travel planning systems, constraint programming surveys
3. Problem Formulation	Formal definition: variables, constraints, objective function
4. System Architecture	Component diagram, data flow, technology choices with rationale
5. Algorithm Design	Insertion heuristic, feasibility propagation, local search — with pseudocode
6. Evaluation	Methodology, metrics, results tables and charts, discussion
7. Conclusions	What was achieved, what was not, future work
Appendix	Full API spec, database schema, test case descriptions

5. What Interviewers and Professors Will Ask

Prepare Exact Answers to These Questions
'Why did you use an insertion heuristic instead of dynamic programming?' — DP is exponential for multi-day VRPTW; insertion is $O(n^3)$ and produces good solutions fast.
'How do you know your solutions are good?' — We measure the optimality gap against OR-Tools on $n \leq 10$ instances. Average gap is X%.
'Does 2-opt always improve the solution?' — No. It can get stuck in local optima. We mitigate this with Or-opt and random restarts.
'Why not use ML for recommendations?' — ML adds complexity without improving the core scheduling problem. Our preference scores already encode user intent.
'What would break with 10,000 POIs?' — The $O(n^2)$ travel matrix would be the bottleneck. We would need spatial clustering to reduce the effective n.
'How does your time-window propagation handle multi-day constraints?' — Each day is independent in our model; inter-day dependencies are only through unscheduled POI carry-over.

6. How to Present Tradeoffs Honestly

 Never Oversell — It Destroys Credibility
Do NOT say 'optimal' — say 'near-optimal on small instances, with measured gaps.'
Do NOT say 'real-time' — say 'generates an itinerary for 15 POIs in under 3 seconds.'
Do NOT say 'scalable' unless you have data — say 'evaluated on instances up to $n=20$; larger instances require the clustering extension described in future work.'
Do NOT say 'AI-powered' — say 'heuristic optimization with local search.'
DO say: 'We deliberately chose engineering simplicity to ensure correctness and measurability.'

7. Demo Scenarios

Prepare three specific, rehearsed demo scenarios. Each must work flawlessly.

Scenario A: Happy Path

User selects 12 POIs in central London across museums, parks, and restaurants. Requests 3 days, 7 hours per day. System returns a feasible itinerary. Presenter highlights arrival times, travel route on map, and day summary cards.

Scenario B: Constrained Edge Case

User selects 5 POIs, several of which close at 17:00. Daily budget is set to 4 hours. System correctly schedules what it can, and clearly explains why 2 POIs were excluded. Presenter shows the violation report.

Scenario C: Evaluation Demo

Presenter switches to the evaluation dashboard. Shows the optimality gap chart. Runs a live OR-Tools comparison on a small instance, demonstrating that the heuristic achieves within X% of optimal in 0.1 seconds vs. OR-Tools taking 30 seconds.

8. Portfolio-Quality GitHub Repository

- Branch strategy: main (stable), dev (active), feature/* (individual features)
- Commit history: meaningful commit messages ('Add feasibility propagation for multi-day plans' not 'fix stuff')
- Issues and milestones: close all open issues before submission
- GitHub Actions CI: run all unit and integration tests on every push to main
- Release tag: v1.0.0 with a release description that summarizes what the system does
- Screenshots and demo GIF in README: essential for visual impact on GitHub profile

9. Resume / Interview Value Added

Portfolio Signal — Phase 4
• End-to-end polished system: extremely rare among student projects — most stop at Phase 2.
• Evaluation-driven claims: any claim in your project is backed by a number. This is what senior engineers do.
• Documented limitations: counterintuitively, honest limitations increase credibility with technical reviewers.
• Reproducible results: a one-command setup and deterministic evaluation pipeline signals production engineering mindset.

10. What Would Make This Phase Elite

- Record a 3-minute screen-recorded demo walkthrough and link it from the README. This is the single highest-ROI action for portfolio value.
- Write a technical blog post (Medium or personal site) explaining the algorithm design decisions. Hiring managers read these and they generate inbound interest.
- Implement a shareable itinerary link: serialize the itinerary to URL parameters so users can share a trip — this is a minor engineering feature but enormously valuable for live demos.

Weekly Breakdown — Phase 4

Week	Focus
Week 33	Frontend: day cards, color-coded polylines, itinerary summary panel
Week 34	Frontend: POI detail drawer, loading states, error handling, responsive layout
Week 35	Itinerary explanation engine: template-based natural language summaries
Week 36	Architecture cleanup: remove dead code, standardize API errors, pin dependencies
Week 37	Documentation: README with architecture diagram, algorithm section, evaluation results
Week 38	Final report writing: Sections 1–4 (Introduction through Architecture)
Week 39	Final report writing: Sections 5–7 (Algorithm, Evaluation, Conclusions)
Week 40	Viva/presentation preparation: slides, rehearse demo scenarios A, B, C
Week 41	GitHub repository cleanup: CI setup, release tag, README screenshots, demo GIF
Week 42	Full dress rehearsal: end-to-end demo run, peer review, final submission prep

Cross-Phase Summary & Dependencies

Phase Dependencies

Each phase has hard dependencies on the previous. Starting Phase 2 before Phase 1's spatial database and API are stable will cause significant rework.

Phase Dependency	Critical Prerequisite
Phase 2 requires Phase 1	PostGIS POI database with valid opening hours, API /pois endpoint, OSRM deployed
Phase 3 requires Phase 2	Feasibility checker validated at 100% correctness, insertion heuristic generating valid itineraries
Phase 4 requires Phase 3	Evaluation results finalized, local search converging, score function tuned

Minimum Viable vs. Stretch Goals

Feature	MVP / Stretch
PostGIS spatial queries with correct indexing	MVP — non-negotiable foundation
Opening hours parsing (80% coverage)	MVP — handle main cases, document failures
OSRM travel-time matrix	MVP — core dependency for all routing
Insertion heuristic with feasibility checking	MVP — the fundamental algorithm
2-opt local search	MVP — minimum optimization to demonstrate improvement
OR-Tools benchmarking ($n \leq 10$)	MVP for evaluation — needed to compute optimality gap
Or-opt local search	Stretch — further improvement, adds evaluation depth
Restart strategies for local search	Stretch — simple improvement with measurable impact
Shareable itinerary links	Stretch — high demo value, low engineering cost
GitHub Actions CI pipeline	Stretch — engineering maturity signal, not core algorithm

Final Correctness Checklist

Before Submission — Verify Every Item

• Every generated itinerary passes the post-generation feasibility checker without exception.
• All unit tests pass on a clean checkout of the repository.
• Evaluation results are reproducible: running eval.py twice produces identical output.
• The README documents every claim with a supporting number or reference.
• The demo works without internet access (OSRM, PostGIS all local).
• Opening hours violations are impossible in the output — the system either schedules correctly or excludes the POI.
• The report's complexity analysis matches the implemented algorithm, not a theoretical variant.

A Final Word on Engineering Philosophy

The measure of this project is not how many features it has — it is how deeply you understand the problem you solved. When a professor or interviewer asks about time-window propagation, or why insertion heuristics work well for VRPTW variants, or what the optimality gap means in practice — your answer should be precise, confident, and grounded in your own experimental results.

That depth of understanding, backed by clean code and honest evaluation, is what transforms a student project into a portfolio centerpiece.

End of Roadmap Document